Model & Backend Notes

Deep-dive notes on the deepfake-detection ML model architecture and a brief overview of the FastAPI backend that serves it. Self-contained: every detail needed to understand the system is described here.

---

1. Big picture

```
uploaded video
      │  OpenCV, ~5 fps, ≤ 100 frames          — backend video processor
      ▼
 frames ──► MTCNN face detect (conf ≥ 0.95)
      │     keep LARGEST box, +20 px margin
      ▼
 face crops ──► CNN classifier ──► p(fake)     — torch, in-process
      │
      ▼
 video score = mean(frame probs)  →  REAL | SUSPICIOUS | HIGH_RISK
      + most suspicious time range             — stored + served to frontend
```

The whole system is **frame-level**: one CNN scores individual face crops. There is no RNN, transformer, or LSTM over time. Temporal structure is collapsed by mean-pooling the per-frame fake probabilities, and a contiguous-run detector finds the most suspicious seconds of the clip.

---

2. ML model architecture (deep dive)

The training and inference pipeline has eight stages, each a script in the `machine-learning/` folder:

1. **Data download** — fetch the training videos.
2. **Preprocessing** — videos → sampled frames → detected/cropped faces.
3. **Dataset loader** — on-disk crops → augmented tensors.
4. **Model definition** — the CNN architecture.
5. **Training** — fine-tuning loop with metrics.
6. **Evaluation** — held-out test metrics and plots.
7. **Inference API** — the function the backend calls.
8. **Comparison** — picks the better of the two trained models.

2.1 Task

Binary classification of a **224×224 RGB face crop** → fake probability in [0, 1]. "Fake" here means the FaceForensics++ (FF++) **Deepfakes c23** manipulation only — that is the scope this project committed to (see §2.9 for why that matters).

2.2 Dataset

- **FaceForensics++ c23** (the "lightly compressed" quality level), taken from the public Hugging Face mirror `bitmind/FaceForensicsC23` because the official download is access-gated.
- Only two of the archive's seven manipulation folders are used:
  - `Real/` — 1,000 authentic videos
  - `Deepfakes/` — 1,000 fake videos
- The raw archive is ~18 GB; the extracted two-class subset is ~5 GB.

2.3 Preprocessing — videos to training crops

FF++ names each fake video after its source: real `033.mp4` and fake `033_097.mp4` are the same person. Two consequences:

1. **Identity-level 70/15/15 split.** Videos are grouped by source identity (`path.stem.split("_")[0]` → identity `033`), identities are shuffled with a fixed seed, and each whole group goes into exactly one of train / val / test. A face never appears in two splits, so there is no identity or near-duplicate-frame leakage between training and evaluation.
2. Per-video frame extraction, all with fixed parameters:
   - **Frame sampling at ~5 fps**: `step = round(src_fps / 5)` , take indices `0, step, 2·step, …` . If that yields more than 50 frames, uniformly thin down to 50 (spread evenly across the clip). Frames are seeked with OpenCV `grab()` so skipped frames are never decoded.
   - **Face detection** with MTCNN (from `facenet-pytorch`), configured `keep_all=True, min_face_size=40` , run under `torch.no_grad()` in batches of 16, GPU-accelerated when available.
   - **Face selection**: keep detections with confidence ≥ 0.95, then pick the **largest box by area** — not the most confident one. This is deliberate: the model learns a particular face-size distribution, so inference must feed it faces selected the same way.
   - **Crop**: expand the box by a 20 px margin (clamped to the frame boundaries; crops under 16 px are rejected), then resize to 224×224 (`INTER_AREA` when downscaling, `INTER_CUBIC` when upscaling).
   - **Output**: JPEG quality 95, saved as `data/processed/{split}/{label}/{video}_{frame}.jpg` , with one row per crop appended to `manifest.csv` (`video, split, label, frame_idx, path`).

Video decoding runs in a thread pool ahead of GPU face detection so the GPU never idles. In this project's run: **99,586 face crops from 2,000 videos** — train 69,767 / val 14,887 / test 14,932, near-perfectly balanced

(49.98 % fake). MTCNN found a face above confidence 0.95 in 99.83 % of sampled frames.

2.4 Model architecture

The architecture is defined in one small function:

```
MODEL_NAMES = {
    "efficientnet_b0": "efficientnet_b0",
    "xception": "legacy_xception",   # timm renamed Chollet's Xception
}

def get_model(name: str, pretrained: bool = True, dropout: float = 0.2) → nn.Module:
    backbone = timm.create_model(MODEL_NAMES[name], pretrained=pretrained, num_classes=0)
    return nn.Sequential(
        backbone,                       # global-pooled feature vector
        nn.Linear(backbone.num_features, 128),
        nn.ReLU(inplace=True),
        nn.Dropout(dropout),            # 0.2
        nn.Linear(128, 1),              # single logit
    )
```

Key design points:

- **Backbone: a CNN from the `timm` model zoo**, one of two candidates:
  - `efficientnet_b0` — ≈ **4.17 M parameters**, 1280-D pooled features
  - `legacy_xception` — ≈ **21.07 M parameters** (timm's name for the original Chollet Xception)

  Two deliberately different-size backbones are trained head-to-head so the cheaper one wins when accuracy ties.
- `num_classes=0` makes timm return the **global-average-pooled feature vector** with no classification head, so the full model is literally `backbone → Linear(1280→128) → ReLU → Dropout(0.2) → Linear(128→1)`.
- The output is **one logit**, trained against `BCEWithLogitsLoss` (logits are numerically stable with that loss); at inference the logit passes through a sigmoid to become a probability in [0, 1].
- The head is a small two-layer MLP on top of the pooled features; it learns the real/fake decision boundary on top of rich ImageNet features.
- **Fine-tuning**: the entire backbone is unfrozen — every layer is updated, no frozen stages, no layer-wise learning rates. Training starts from ImageNet weights; the head starts from default PyTorch init.

- **Input normalization** is ImageNet's: mean `(0.485, 0.456, 0.406)`, std `(0.229, 0.224, 0.225)`, applied after resizing every crop to 224×224.

Why this shape? It is the classic deepfake baseline: a strong ImageNet CNN fine-tuned on face crops. Modern manipulation artifacts are largely visible in single frames (blending seams, resolution mismatch, warping), so a frame-level CNN gets very far — the results in §2.7 confirm it. Temporal modeling is deliberately out of scope (§2.9).

2.5 Loader & data augmentation

- The dataset class reads its split's rows from `manifest.csv`, opens each crop path, converts to RGB, and yields `(image_tensor, target)` where target is float `0.0` (real) or `1.0` (fake).
- Common transform applied to every sample: `Resize((224,224)) → ToTensor() → ImageNet Normalize`.
- **Train-only augmentation** — kept mild so faces stay recognizable:
  - `RandomHorizontalFlip(p=0.5)`
  - `RandomAffine(degrees=10, translate=(0.05, 0.05))` — ±10° rotation, ±5 % translation
  - `ColorJitter(brightness=0.2, contrast=0.2)`
- DataLoaders: batch size 64 (EfficientNet) or 32 (Xception), 4–8 worker processes, `pin_memory` on CUDA. Only the training loader shuffles and drops the last (partial) batch.

2.6 Training

Hyperparameters (identical for both models except batch size):

Setting	Value
Optimizer	Adam, lr 1e-4, weight decay 1e-5
Loss	`BCEWithLogitsLoss` with `pos_weight` = train_real / train_fake ≈ 1.0029
Batch size	64 (EfficientNet-B0) / 32 (Xception)
Max epochs	15
Early stopping	on **val ROC-AUC**, patience 5
Gradient clipping	max norm 1.0
Precision	AMP fp16 (`torch.autocast` + `GradScaler`), CUDA only
Memory format	`channels_last` on CUDA
Seed	42 (torch, numpy, random, cuda)

Notes on the loop:

- `pos_weight` compensates for the tiny class imbalance (≈1.003 here, i.e. nearly neutral because the crop dataset is balanced).

- `optimizer.zero_grad(set_to_none=True)`; forward pass under `autocast`; `scaler.scale(loss).backward()`; `scaler.unscale_()`; clip gradients; then `scaler.step()` + `scaler.update()`.
- Each epoch evaluates train and val: mean loss, accuracy at the 0.5 threshold, and **ROC-AUC** via `sklearn.metrics.roc_auc_score` (NaN-guarded if a validation split happens to contain a single class).
- **Model selection is by best val ROC-AUC** — not loss or accuracy. Whenever it improves, the `state_dict` is saved as `best.pth` in `checkpoints/<model>_<timestamp>/`. If five epochs pass without an AUC gain, training stops early.
- Alongside the weights, a `config.json` records the full training state: model name, every CLI argument, image size, normalization constants, class mapping (real = 0, fake = 1), per-split class counts, `pos_weight`, best epoch, best val metrics, device, GPU name, and whether AMP was used. This makes every run exactly reproducible and lets inference rebuild the architecture from disk alone.
- Training hardware: NVIDIA GeForce RTX 4060 Laptop GPU (8 GB), 16-core CPU, 15 GB RAM. End-to-end wall time for both runs (download excluded): ~12 min preprocessing, 24 min EfficientNet-B0, 42 min Xception.

2.7 Results (held-out test split, 0.5 threshold)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	ms/img	Best epoch	Train time
EfficientNet-B0	0.9912	0.9887	0.9938	0.9912	0.9989	2.76	8 (stop @13)	24 min
Xception	0.9858	0.9919	0.9796	0.9857	0.9990	2.08	6 (stop @11)	42 min

(Reported over 14,932 test crops at threshold 0.5; latency is measured as 100 batched forward passes on a 1×3×224×224 tensor after warm-up, under AMP.)

Both clear the ≥ 0.90 ROC-AUC exit criterion by a wide margin. The ROC-AUC difference between the two models is 0.0001 — noise — so the comparison ranks them by **F1 at threshold 0.5** and picks **EfficientNet-B0** as the production checkpoint:

- essentially identical accuracy at **5× fewer parameters** (4.17 M vs 21.1 M),
- half the training time,
- errs toward recall (precision 0.9887 / recall 0.9938), i.e. it catches more fakes at the cost of a few more false alarms.

Xception is the more conservative alternative: higher precision (0.9919), lower recall (0.9796) — it misses more fakes but raises fewer alarms on real video.

Threshold behavior: the backend's risk bands are 0.4 and 0.7 (see §3.3). Frame-level performance is essentially flat across that whole band — at t = 0.41: precision 0.9880 / recall 0.9949 / F1 0.9915 / accuracy

0.9914; at t = 0.70: precision 0.9905 / recall 0.9910 / F1 0.9908 / accuracy 0.9908 — so the default cutoffs needed no retuning.

2.8 Inference API — the contract the backend consumes

```
model = load_model("machine-learning/checkpoints/efficientnet_b0_20260901_204509")
prob  = predict(model, face_crop_rgb_ndarray)   # 0.0-1.0 fake probability
```

- `load_model(checkpoint_dir)` reads the run's `config.json`, rebuilds the exact architecture with `get_model(config["model"], pretrained=False)` — weights come from disk, so no ImageNet download — then loads `best.pth` with `load_state_dict`, moves the model to CUDA if available (else CPU), and sets `.eval()`. It attaches `.device` and `.config` attributes to the module.
- `predict(model, x)` accepts a file path, a PIL image, an RGB `uint8` numpy array (a 2-D grayscale array is stacked to 3 channels; non-uint8 arrays are clipped), or a torch tensor — **or a list of any of those**, which is scored in a single batched forward pass. That list mode is what makes whole-video scoring fast.
- Preprocessing inside `predict` is exactly the eval-time transform: resize to 224×224, ImageNet normalize — no augmentation, no random flips.
- The return value is `sigmoid(logit)` squeezed to a float in [0, 1] (a list of floats for list input).
- Model artifacts are plain PyTorch (`best.pth` state_dict + `config.json`); there is no ONNX/TensorRT export.
- A smoke test scores a sample of real and fake test crops and asserts the class means land on the correct side of 0.5, plus that single-image and batched predictions agree within 1e-4. Observed means: real ≈ 0.0099, fake ≈ 0.9862.

2.9 Limitations & extension paths

- **One manipulation family.** The model was trained on FF++ Deepfakes only; expect weaker generalization to other methods (Face2Face, FaceSwap, NeuralTextures) or in-the-wild fakes. Broadening the training data is a one-line change (extract more of the seven archive folders) followed by re-running preprocessing and training.
- **No temporal modeling.** A fake that only betrays itself over time (frame-rate mismatch, flicker, temporal blending) is missed by a frame-level CNN. Adding a sequence model — LSTM over per-frame features, a transformer, or a 3D CNN — is the natural next phase; the backend already stores the per-frame score timeline, which is exactly the input such a model needs.
- **Face detection is a hard dependency.** Videos where no frame yields a face at confidence ≥ 0.95 (with a ≥ 16 px crop after margin) cannot be scored and error out.
- **Clean-source bias.** Results are on compressed-but-clean FF++ c23 videos; heavily re-encoded social-media uploads will be harder.

---

3. Backend (brief)

A single **FastAPI** service (Python, uvicorn, port 8000) loads the ML checkpoint **in-process** — there is no separate model server and no HTTP hop between the API and the model. The web UI is a separate Next.js app (port 3000) that talks to the API over CORS.

3.1 Layout

- `backend/main.py` — app factory: registers routers, CORS middleware, creates database tables and the upload directory at startup, then loads the model.
- `backend/config.py` — pydantic-settings, reads `backend/.env`. Key knobs: `NEON_DB_URL` (else local SQLite `backend/app.db`), `MODEL_DIR` (checkpoint to load), `UPLOAD_DIR`, risk cutoffs `RISK_SUSPICIOUS = 0.4` and `RISK_HIGH = 0.7`, `FRAME_THRESHOLD = 0.7`, `MAX_UPLOAD_MB = 200`, `CORS_ORIGINS`.
- `backend/models.py` + `backend/database.py` — SQLAlchemy 2.0. One table, `analyses`: id, filename, storage_path, fake_probability, status, suspicious_start/end, frame_scores (JSON), created_at. Schema upgrades are handled by a lightweight auto-ALTER helper (no Alembic).
- `backend/services/detector.py` — thin wrapper around the ML inference API (adds `machine-learning/` to `sys.path` and imports `inference`).
- `backend/services/video_processor.py` — the video → verdict pipeline.
- `backend/routes/` — HTTP endpoints.

3.2 Endpoints

Method	Path	Purpose
GET	`/health`	`{"status": "ok", "model_loaded": bool}` — `model_loaded` is false when the detector runs in stub mode
POST	`/analyze`	multipart upload (`.mp4` / `.avi` / `.mov`, ≤ 200 MB, else 413) → full pipeline → 201 response
GET	`/analysis/{id}`	one stored analysis (same shape as the analyze response)
GET	`/analysis/{id}/video`	streams the stored source video back with Range support, for in-browser playback
GET	`/history?limit=`	recent analyses, newest first (limit 1–500, default 100)

Response shape: `{id, filename, fake_probability, status, suspicious_start, suspicious_end, frame_scores: [{timestamp, fake_probability}], created_at, has_video}` with `status ∈ REAL | SUSPICIOUS | HIGH_RISK`. Errors: 400 bad extension or empty file, 413 too large, 422 unreadable video / no faces, 404 unknown id or missing video file.

3.3 Video pipeline — what happens on upload

1. **Frame extraction** (OpenCV): read frame count and fps, sample ~5 fps capped at 100 frames, using the same two-stage sampling as training preprocessing — so inference frames are drawn the way training frames were. Unreadable files raise an error.
2. **Face detection** (MTCNN, lazy singleton, `keep_all=True, min_face_size=40`): per frame, keep the **largest** face with confidence ≥ 0.95, cropped with a 20 px margin — the identical rule that produced the training crops. Frames with no qualifying face are skipped; if no frame has a face, the video errors out.
3. **Scoring**: all crops are resized to 224×224, normalized, and passed in **one batched forward pass** to the model (the detector's list mode). The result is a per-frame timeline `[{timestamp, fake_probability}]`.
4. **Aggregation**: the video's `fake_probability` is the **mean of the frame probabilities**.
5. **Risk status**: `< 0.4` → REAL; `0.4–0.7` → SUSPICIOUS; `> 0.7` → HIGH_RISK.
6. **Suspicious region**: the longest contiguous run of frames with probability ≥ 0.7 (needs ≥ 3 consecutive frames); its first and last frame timestamps become `suspicious_start` / `suspicious_end`.

The result row — verdict plus the full frame timeline — is persisted, so the frontend can replay per-frame probabilities later without re-analyzing.

3.4 Detector stub mode

If the ML module or checkpoint is missing at startup, the detector logs a warning and every `predict` returns a fixed **0.5**. Routes and the frontend still work during development without a trained model; `/health` exposes `model_loaded` so you can tell which mode you are in. `MODEL_DIR` may point at one checkpoint run directory or at the parent `checkpoints/` folder, in which case the most recently written run is auto-selected. The deployment pins `MODEL_DIR` to the EfficientNet-B0 run.

3.5 Storage, uploads & deployment

- **Database**: Neon Postgres when `NEON_DB_URL` is set; otherwise a local SQLite file. Both paths pass the project's smoke test.
- **Uploads**: streamed to disk in 1 MB chunks (memory-safe enforcement of the 200 MB cap); deleted on failure, kept on success so the video can be replayed. On serverless deployments the filesystem is ephemeral, so stored videos may not survive restarts — `has_video` reflects that.
- **Container image** (repo-root Dockerfile): installs **CPU-only** torch (`torch==2.6.0+cpu` from the PyTorch CPU index — the model runs fine on CPU, just slower), pins pandas (the ML dataset module imports it at module level), and installs `facenet-pytorch` with `--no-deps` because its package metadata pins an old `torchvision` that conflicts with the current wheels (the MTCNN code itself is compatible). Runs as uid 1000; the command is `uvicorn backend.main:app --port ${PORT:-7860}`, which honors the platform's injected port (e.g. 8080 on Cloud Run, 7860 on Hugging Face Spaces).
- **Dev launcher**: a repo-root script creates a `.venv`, installs dependencies, starts the backend on :8000 and the frontend dev server on :3000 (auto-stepping busy ports), polls `/health` until ready, and tees logs to a `logs/` folder.

---

4. One-paragraph summary

Phase 1 trains a **frame-level binary CNN classifier** — an ImageNet fine-tuned EfficientNet-B0 or Xception backbone with a 128-D ReLU head ending in a single logit — on 224×224 MTCNN face crops from 2,000 FaceForensics++ Deepfakes videos, split 70/15/15 by source identity to prevent leakage. It is trained with binary cross-entropy (class-weighted), Adam at lr 1e-4, AMP fp16, gradient clipping, and early stopping on validation ROC-AUC; both models reach ≈0.99 test accuracy and ≈0.999 AUC, and the 4.17 M-parameter EfficientNet-B0 is the shipped checkpoint. Phase 2 wraps it in a FastAPI service: uploaded videos are sampled at 5 fps, faces are cropped with the same MTCNN rule used for training, all crops are scored in a single batch, and the video verdict is the **mean frame probability** mapped to REAL / SUSPICIOUS / HIGH_RISK via 0.4 / 0.7 cutoffs, with the longest contiguous ≥ 0.7 run reported as the suspicious time range. There is no temporal model yet — that is a documented future extension.